

✓ 1. Titre du Mémoire

Détection Autonome et Distribuée d'Anomalies (comportements suspects ou signes d'intrusion) dans les Journaux Systèmes et Réseaux à l'aide d'Agents Intelligents Multi-Tâches

Etudiant : Andy Aaron Bitati (Tech-1, GI)

Date : 14 Avril 2025

✓ 2. Superviseurs

- Directeur : Prof. Kyamakya
- Co-superviseurs :
 - Ass. Toms Mugisha
 - Ass. Vianney Kambale
 - Ing. Jean-Claude Mudilu (+243 814 440 110)
 - Ing. Gilbert Mbula

✓ 3. Objectifs principaux du Mémoire (7 objectifs)

1. **Comprendre les types et structures de journaux (logs) système/réseau**
→ Identifier les formats courants (syslog, Windows Event Log, Apache logs, etc.)
2. **Analyser les techniques classiques et modernes de détection d'anomalies**
→ Comparer les approches statistiques, heuristiques, et basées IA
3. **Développer une architecture distribuée d'agents IA spécialisés**
→ Chaque agent aura une tâche spécifique : parsing, corrélation, détection, etc.
4. **Intégrer un mécanisme d'apprentissage pour la détection adaptative**
→ Utiliser des modèles IA légers (Autoencoders, Isolation Forest, LSTM)
5. **Concevoir un tableau de bord pour visualiser les alertes et anomalies**
→ Interface web simple pour affichage et interaction avec le système
6. **Tester le système sur des logs simulés et réels**
→ Utiliser des jeux de données publics ou des journaux capturés localement
7. **Évaluer la performance du système (précision, rappel, F1-score, latence)**
→ Mesure comparative avec des outils classiques (fail2ban, OSSEC, etc.)

DETAILS :

Objectif 1 : Identifier, catégoriser et structurer les différents types de journaux systèmes et réseaux pertinents pour l'analyse de sécurité

Cet objectif vise à dresser un inventaire complet des types de logs utilisés en cybersécurité, notamment les journaux systèmes (Linux, Windows), applicatifs (serveurs web, bases de données), et réseaux (pare-feux, IDS/IPS). Il s'agit également d'étudier leur format, leur fréquence d'émission, leur structure syntaxique et sémantique, afin de préparer leur traitement automatique.

Résultats attendus :

- Taxonomie des logs courants en cybersécurité.
 - Création d'un module de parsing multi-format (syslog, JSON, CSV, etc.).
 - Normalisation des événements pour traitement unifié.
-

Objectif 2 : Étudier, comparer et choisir des approches classiques et intelligentes pour la détection d'anomalies dans les journaux

Analyser les méthodes existantes (statistiques, seuils, règles expertes, apprentissage non supervisé/supervisé, deep learning léger) et sélectionner les plus appropriées aux environnements à ressources limitées. Cet objectif servira à déterminer les bases méthodologiques du système.

Résultats attendus :

- Grille comparative des techniques d'anomalie.
 - Justification du choix de modèles légers (e.g. Isolation Forest, Autoencoder).
 - Définition des métriques de performance (précision, rappel, F1-score).
-

Objectif 3 : Concevoir une architecture distribuée et modulaire basée sur des Agents IA spécialisés pour la surveillance et l'analyse des logs

Il s'agit ici de définir les rôles des différents agents (collecteur, parseur, analyste, corrélateur, visualiseur, etc.), leurs flux de communication, et leur coordination. Cette architecture distribuée permettra la scalabilité du système et sa résilience face à des volumes élevés de logs.

Résultats attendus :

- Diagrammes UML d'architecture multi-agent.
 - Définition des protocoles d'interaction (pub/sub, sockets, REST).
 - Élaboration d'un prototype modulaire prêt à implémenter.
-

Objectif 4 : Intégrer des modèles d'intelligence artificielle légers pour la détection d'anomalies dans les flux de logs en temps quasi-réel

Chaque agent analytique doit intégrer un ou plusieurs modèles IA entraînés ou adaptables. L'accent est mis sur les modèles légers capables de fonctionner sur des machines non spécialisées (autoencoders, clustering, réseaux récurrents légers, etc.).

Résultats attendus :

- Intégration de modèles IA (Sklearn, Keras, etc.).
 - Capacité d'auto-ajustement selon les nouvelles données.
-

- Benchmark des performances en conditions simulées.
-

 Objectif 5 : Développer une interface utilisateur visuelle (dashboard) permettant d'explorer les anomalies détectées de façon compréhensible

Un tableau de bord simple mais efficace est essentiel pour les administrateurs. Il devra afficher les logs, les alertes d'anomalies, les statistiques temporelles, et permettre une interaction minimale (reclassement, étiquetage, visualisation par type/source de log).

Résultats attendus :

- Développement d'une interface web (Streamlit/Flask).
 - Graphiques en temps réel (courbes, heatmaps, timelines).
 - Interaction avec les agents (validation/rejet d'alertes).
-

 Objectif 6 : Tester le système sur des jeux de données variés (logs simulés, journaux réels, attaques connues) afin d'évaluer sa robustesse

Cet objectif validera la robustesse, la précision, et la valeur ajoutée du système multi-agents dans des cas pratiques : logs réels d'OS, journaux de réseaux simulés avec anomalies connues, et situations d'intrusion simulée (e.g. brute force, port scanning, injection).

Résultats attendus :

- Jeux de données importés et nettoyés.
 - Injection d'événements anormaux ou d'attaques simulées.
 - Résultats comparés à des outils standards (e.g. fail2ban, Wazuh).
-

 Objectif 7 : Évaluer globalement les performances, la latence, la précision et la capacité d'extension du système multi-agents IA développé

En fin de projet, il faudra évaluer la pertinence globale de l'approche selon des critères bien définis : temps de traitement, taux de détection, scalabilité, modularité, etc. Cette évaluation permettra de juger de la faisabilité d'un déploiement en conditions réelles.

Résultats attendus :

- Tableau complet de performances (par module, par agent).
 - Évaluation de la charge système (CPU/RAM).
 - Recommandations pour industrialisation ou amélioration future.
-

✓ 4. Méthodologie Générale (par objectif)

Objectif	Méthodologie
1. Logs	Étude documentaire + parsing avec regex, logstash
2. Tech. d'anomalie	Évaluation de modèles : z-score, clustering, Autoencoder
3. Architecture	Utilisation d'un orchestrateur Python (FastAPI) pour agents
4. IA adaptative	Implémentation d'auto-apprentissage (ML semi-supervisé)
5. Visualisation	Développement d'un front-end en Streamlit ou Flask
6. Testing	Jeux de données publics (e.g. HDFS log, BGL, DARPA 98)
7. Évaluation	Utilisation de métriques standard + comparaison avec baseline

DETAILS:

◆ Objectif 1 : Identifier, catégoriser et structurer les différents types de journaux systèmes et réseaux pertinents pour l'analyse de sécurité

🔧 Méthodologie proposée :

- Recherche documentaire :**
 - Étudier les différents types de journaux couramment utilisés : `syslog`, `journalctl` (Linux), `event logs` (Windows), `Apache logs`, `firewall logs`, `IDS/IPS logs`, etc.
 - Analyser les normes de structuration des logs (RFC 5424 pour Syslog, JSON logs, formats CSV, etc.).
- Collecte de données :**
 - Capturer des logs réels (environnement Linux ou Windows en local, outils de simulation réseau).
 - Télécharger des jeux de données publics (BGL, HDFS, DARPA datasets).
- Analyse et normalisation :**
 - Utiliser des scripts Python pour extraire les éléments clés : timestamp, source, niveau de sévérité, message.
 - Concevoir un schéma de normalisation pour un traitement homogène.
- Catégorisation :**
 - Identifier les familles d'événements : erreurs système, tentatives de connexion, déni de service, etc.

◆ Objectif 2 : Étudier, comparer et choisir des approches classiques et intelligentes pour la détection d'anomalies dans les journaux

🔧 Méthodologie proposée :

1. **État de l'art comparatif :**
 - Dresser une grille comparative des méthodes traditionnelles (règles, seuils, entropie) et modernes (machine learning, deep learning).
 2. **Implémentation expérimentale :**
 - Implémenter un échantillon de modèles légers :
 - Méthodes statistiques : z-score, IQR, histogramme.
 - Méthodes IA : Isolation Forest, k-Means, Autoencoder, LSTM.
 3. **Évaluation sur logs simulés :**
 - Appliquer les méthodes sur un dataset avec anomalies injectées manuellement (ex. injection d'erreurs, d'attaques simulées).
 4. **Critères de sélection :**
 - Choisir les meilleurs modèles selon : **précision, temps de traitement, capacité d'adaptation, et consommation mémoire.**
-

◆ Objectif 3 : Concevoir une architecture distribuée et modulaire basée sur des Agents IA spécialisés pour la surveillance et l'analyse des logs

🔧 Méthodologie proposée :

1. **Définition fonctionnelle des agents IA :**
 - Agent 1 : Collecte et parsing des logs.
 - Agent 2 : Prétraitement et normalisation.
 - Agent 3 : Détection d'anomalies.
 - Agent 4 : Corrélation contextuelle.
 - Agent 5 : Visualisation des alertes.
 2. **Conception de l'architecture globale :**
 - Utilisation de **diagrammes UML** (composants, séquences, communication).
 - Communication inter-agents via **REST API, MQTT, ou sockets Python.**
 3. **Prototypage des agents en environnement local :**
 - Déploiement de prototypes sous forme de services Python (FastAPI, Flask).
 - Utilisation d'un orchestrateur léger (e.g., Docker Compose ou même un bus de messages simple).
 4. **Validation de la communication entre agents :**
 - Tests d'intégration entre agents avec simulation de flux de logs en temps réel.
 - Surveillance de la stabilité du système distribué.
-

◆ Objectif 4 : Intégrer des modèles d'intelligence artificielle légers pour la détection d'anomalies dans les flux de logs en temps quasi-réel

🔧 Méthodologie proposée :

1. **Choix de modèles IA adaptés aux environnements contraints :**
 - Sélectionner des modèles **peu gourmands en ressources** mais efficaces :
 - *Isolation Forest*
 - *One-Class SVM*
 - *Autoencoder simple (avec peu de neurones)*
 - *LSTM compressé ou GRU léger* pour logs séquentiels.
2. **Prétraitement des données :**
 - Tokenisation des messages logs si nécessaire.
 - Encodage (TF-IDF, one-hot, embeddings simples).
3. **Entraînement supervisé / non-supervisé :**
 - Entraîner les modèles sur des données “propres” pour qu'ils apprennent le comportement normal.
 - Tester sur des données contenant des anomalies injectées ou réelles.
4. **Intégration avec les agents IA :**
 - Chaque agent dispose de son propre modèle ou interagit avec un micro-service de prédiction.
 - Les agents adaptent leur seuil de décision selon les retours du système.
5. **Suivi de performance en streaming :**
 - Déploiement de tests de détection en temps réel avec simulation de flux via watch, log-generator, ou log replay.

◆ Objectif 5 : Développer une interface utilisateur visuelle (dashboard) permettant d'explorer les anomalies détectées de façon compréhensible

🔧 Méthodologie proposée :

1. **Spécification des besoins utilisateurs :**
 - Identifier les indicateurs clés : type d'anomalie, horodatage, source, gravité, fréquence.
 - Définir les vues utiles : vue globale, par source, par agent, par type.
2. **Choix des outils de visualisation :**
 - Utiliser des frameworks simples comme **Streamlit**, **Flask**, ou **Dash**.
 - Pour les graphes interactifs : **Plotly**, **Bokeh**, ou **Chart.js** via Flask.
3. **Conception du dashboard :**
 - Layout responsive avec filtre par source, niveau, type d'événement.
 - Section d'analyse temporelle (heatmap, timeline).
 - Possibilité de valider/rejeter les anomalies détectées.
4. **Connexion avec les agents IA :**
 - Dashboard connecté à une base temporaire alimentée par les agents.
 - Rafraîchissement en temps réel ou toutes les X secondes.
5. **Tests utilisateurs :**

- Recueillir des feedbacks sur la lisibilité, l'ergonomie et les performances.

◆ **Objectif 6 : Tester le système sur des jeux de données variés (logs simulés, journaux réels, attaques connues) afin d'évaluer sa robustesse**

🔧 **Méthodologie proposée :**

1. **Sélection et préparation des jeux de données :**
 - Utiliser des datasets publics :
 - *HDFS log dataset*
 - *BGL log (BlueGene/L)*
 - *DARPA IDS dataset*
 - *OpenSyslog repo* sur GitHub
 - Générer localement des logs de Linux, Apache, SSH, IDS (via virtualisation ou log-generator).
2. **Simulation d'attaques connues :**
 - Générer ou injecter manuellement :
 - brute force
 - privilege escalation
 - DoS simulé
 - injection de commandes
3. **Exécution de scénarios contrôlés :**
 - Définir des scénarios avec variation du volume, type, et fréquence d'événements.
4. **Analyse comparative :**
 - Comparer les résultats du système avec ceux d'outils comme *fail2ban*, *OSSEC*, *Wazuh*.
5. **Évaluation qualitative :**
 - Demander à un expert ou un administrateur d'évaluer les alertes détectées.

◆ **Objectif 7 : Évaluer globalement les performances, la latence, la précision et la capacité d'extension du système multi-agents IA développé**

🔧 **Méthodologie proposée :**

1. **Définition des métriques de performance :**
 - Précision, rappel, F1-score.
 - Taux de faux positifs / faux négatifs.
 - Temps de réponse moyen (latence).
 - Utilisation CPU/mémoire par agent.
2. **Tests en conditions réalistes :**
 - Simulation de fonctionnement sur plusieurs heures avec différents volumes de logs.
 - Tests de montée en charge (stress tests).

3. **Évaluation de la scalabilité :**
 - Ajouter progressivement des sources de logs (simuler 5, 10, 50 machines).
 - Observer la stabilité du système et la coopération entre agents.
4. **Tests de robustesse :**
 - Simuler l'arrêt d'un agent → observer la résilience de l'architecture.
 - Injection de logs corrompus → observer la tolérance aux erreurs.
5. **Analyse finale et recommandations :**
 - Comparaison avec les performances attendues.
 - Propositions d'amélioration ou de déploiement à grande échelle.



5. Innovations spécifiques du mémoire

- **Architecture multi-agents coopérants** pour une surveillance distribuée.
- **Apprentissage automatique intégré** pour s'adapter aux journaux en évolution.
- **Visualisation des anomalies en temps réel** avec classification contextuelle.
- **Système modulaire et léger** pour PME et universités sans SOC.

DETAILS:



1. Système Multi-Agent IA Décentralisé pour la Détection de Menaces

Logiques

Le mémoire introduit une **architecture multi-agent inédite** dans le contexte de l'analyse des journaux (logs) système et réseau. Chaque agent est **spécialisé** (collecte, prétraitement, détection, visualisation) et peut fonctionner **autonument ou en coopération**, ce qui permet :

- Une **meilleure scalabilité** (facile à adapter à de nouveaux types de logs).
- Une **résilience accrue** (un agent peut tomber sans arrêter le système).
- Une **répartition de charge** sur plusieurs machines ou processus.



Innovation : modèle décentralisé inspiré de l'architecture cognitive distribuée.



2. Intégration d'IA légère dans chaque agent pour une détection adaptative et temps réel

Contrairement aux systèmes centralisés utilisant des algorithmes lourds, ce mémoire met en œuvre des modèles **IA légers et localement déployables** (Isolation Forest, Autoencoder, k-Means, etc.), compatibles avec des **environnements à faibles ressources** (serveurs simples, Raspberry Pi, machines virtuelles).

- Apprentissage **semi-supervisé** ou **non supervisé** intégré.

- Agents IA **auto-ajustables** à partir du comportement normal appris.

⚡ **Innovation** : intelligence distribuée, autonome et dynamique.

3. Agents avec capacités de raisonnement et interaction (vers des agents cognitifs)

Certains agents sont conçus pour **corrélér** les événements anormaux entre eux (ex : plusieurs échecs SSH suivis d'une commande root). Ce raisonnement de haut niveau s'apparente à une **capacité cognitive**, et ouvre la voie à une **détection plus contextuelle et interprétable**.

- Base de règles contextuelles simples intégrée (logique floue, logique d'alarme).
- Agents pouvant partager leur interprétation avec un agent "superviseur".

⚡ **Innovation** : transition vers des **agents IA cognitifs hybrides**.

4. Modularité extrême du système : plug & play par type de log ou de rôle

Chaque agent peut être activé/désactivé sans affecter l'ensemble du système. Il devient ainsi possible de :

- Remplacer un modèle IA par un autre sans redéployer tout le système.
- Ajouter des agents pour de nouveaux types de journaux (ex : email logs).
- Déployer un sous-système dédié (ex : seulement détection SSH).

⚡ **Innovation** : système **évolutif et modulaire** à faible coût de maintenance.

5. Tableau de bord intelligent intégré et interactif

Le système ne se contente pas de produire des alertes : il **visualise, explique et contextualise** les anomalies détectées, à l'aide d'une **interface web dynamique** permettant :

- Une interaction en temps réel (valider/rejeter une alerte).
- Une visualisation filtrée par source/type/gravité.
- Une journalisation complète des décisions de l'utilisateur (audit trail).

⚡ **Innovation** : cybersécurité **interprétable et collaborative**, en temps réel.



6. Complémentarité avec les outils classiques : renforcement des systèmes existants

Le système est conçu pour fonctionner **en complément des outils existants** tels que fail2ban, Wazuh, ou OSSEC, apportant :

- Une **intelligence adaptative** sur les anomalies faibles ou nouvelles.
- Une **capacité de priorisation** des alertes, basée sur le contexte.
- Une **exploitation plus profonde des journaux** ignorés ou non analysés par défaut.



Innovation : hybridation entre IA moderne et sécurité opérationnelle classique.



6. Importance pratique du thème

- Répond au **besoin urgent d'automatisation de la cybersécurité**, notamment dans les environnements à ressources limitées.
- Permet de **détecter des menaces internes et externes** à partir de journaux existants.
- Application directe dans **les universités, entreprises locales, administrations**.
- Peut servir de **base à un produit commercial de cybersécurité IA open source**.

DETAILS :



1. Réponse directe à la pénurie d'experts en cybersécurité

Dans de nombreux contextes, notamment dans les pays en développement ou les petites structures (PME, universités, collectivités), il n'existe pas toujours d'experts dédiés à la cybersécurité. Ce thème propose une **automatisation intelligente** des audits via des agents IA, rendant possible un **autocontrôle de sécurité** même sans ressources humaines expertes.

Impact concret :

- Réduction de la dépendance aux consultants externes.
- Autonomie accrue dans la gestion de la sécurité.



2. Amélioration proactive de la détection des menaces grâce aux journaux sous-exploités

La majorité des attaques ou anomalies laissent des traces dans les logs. Toutefois, ces données sont **peu exploitées manuellement** faute de temps ou d'outils adaptés. Le système proposé transforme les journaux bruts en **informations exploitables en temps réel**, aidant à identifier les **intrusions, comportements anormaux, et défaillances internes**.

Impact concret :

- Surveillance continue sans coût d'exploitation élevé.
 - Détection d'anomalies complexes invisibles aux règles statiques.
-



3. Système adaptable à différentes échelles et contextes d'infrastructure

Grâce à son architecture modulaire et à l'usage d'agents autonomes, le système peut être **déployé partiellement ou totalement**, selon les besoins et capacités d'une organisation. Il peut être :

- déployé sur un seul serveur,
- étendu à plusieurs machines,
- ou intégré dans une solution cloud/hybride.

Impact concret :

- Adaptation facile pour une entreprise, une école, une administration locale.
 - Possibilité d'évolution vers un SOC (Security Operations Center) léger.
-



4. Optimisation des processus de réponse et réduction du temps moyen de détection (MTTD)

En automatisant l'analyse, la détection, et même la **classification des anomalies**, ce projet réduit significativement le **temps de réaction** face à un incident. Cela renforce la **résilience organisationnelle** face aux cybermenaces croissantes.

Impact concret :

- Meilleur taux de réponse aux incidents.
 - Prévention renforcée contre les attaques à répétition.
-



5. Complément idéal aux outils de sécurité existants (antivirus, pare-feux, IDS/IPS)

L'approche ne cherche pas à remplacer les outils existants, mais à **les enrichir**, en apportant une **couche intelligente de corrélation et d'analyse** des journaux qu'ils génèrent. Cela permet par exemple :

- d'analyser les logs antivirus pour détecter des tendances suspectes,
 - de corréler des logs réseau avec des logs système,
 - ou de prioriser les alertes générées.
-

Impact concret :

- Synergie entre les différents systèmes de sécurité.
 - Réduction des faux positifs et alertes non traitées.
-



6. Potentiel d'industrialisation et de transfert technologique local

Le système développé peut être **transformé en solution logicielle réutilisable** (open-source ou commerciale) pour les structures locales. Il peut donner lieu à :

- une startup spécialisée en cybersécurité basée sur IA,
- une plateforme d'audit pour écoles/universités,
- un service packagé pour les TPE et ONG.

Impact concret :

- Création de valeur locale dans le domaine critique de la cybersécurité.
- Opportunité de formation, innovation et entrepreneuriat.

✓ 7. Ressources utiles

🔧 Frameworks & Outils

- Python, Pandas, Scikit-learn, TensorFlow/PyTorch (light)
- Logstash, Elasticsearch, Kibana, Grafana
- Scapy ou Wireshark pour l'analyse réseau

🔍 Datasets

- HDFS Log Dataset
- BGL System Logs
- DARPA Intrusion Detection Dataset
- RALIC Syslog Dumps

📖 Articles et GitHub

- GitHub : [logpai/loglizer](#)
- GitHub : [AI-cyber-agent](#)
- Survey: *Deep Learning for Log Anomaly Detection: A Survey*, ACM 2022
- Article : *A Comparative Study of Anomaly Detection Techniques in Logs*, IEEE Access

DETAILS:

🧠 1. Bibliothèques et Frameworks d'Intelligence Artificielle

Outil	Utilité principale
Scikit-learn	Modèles d'IA légers : Isolation Forest, k-Means, PCA
TensorFlow / Keras	Création de modèles Autoencoder ou LSTM
PyTorch	Développement de modèles IA personnalisés et flexibles
River	Apprentissage en flux (online learning) pour les logs
XGBoost / LightGBM	Détection avancée si supervision possible

👤 2. Outils de traitement des logs et pré-analyse

Outil	Fonction
Logstash	Parsing, transformation et routage de logs (format JSON, CSV, etc.)
Fluentd	Collecte et agrégation de logs multi-source
Syslog-ng	Capture et centralisation des logs Linux
ElasticSearch + Kibana	Stockage, requêtage et visualisation des événements
Graylog	Alternative à ELK, avec interface graphique légère



3. Frameworks pour Agents IA et Systèmes Multi-Agents

Framework	Description
Python Agent DEAP / Mesa	Simulation et coordination d'agents dans des systèmes distribués
FastAPI	Construction d'agents intelligents exposés via API REST
Celery + Redis	Orchestration des tâches entre les agents
SuperAgent (GitHub)	Construction d'agents autonomes guidés par IA
Autogen (Microsoft)	Création d'agents coopérants (LLM + tâches spécialisées)



4. Jeux de Données Publics (Logs & Anomalies)

Dataset	Description
HDFS Log Dataset (UC Berkeley)	Journaux de système Hadoop avec anomalies annotées
BGL Logs (BlueGene/L)	Logs système haute performance, avec erreurs identifiées
DARPA IDS 1999	Logs réseau + attaques connues (telnet, ftp, etc.)
OpenSyslog Dataset	Ensemble de logs Linux/UNIX ouverts
LogPai Datasets	Variétés de journaux applicatifs avec anomalies détectables

🔗 Portail utile : <https://github.com/logpai/loghub>



5. Articles et lectures scientifiques clés

- "A Survey on Log Anomaly Detection using Machine Learning" – ACM Computing Surveys, 2022.
- "LogRobust: A Self-Supervised Framework for Robust Log Anomaly Detection" – IEEE TNNLS, 2021.
- "Logsy: AI Agents for Cybersecurity Audit through System Log Analysis" – Journal of Cybersecurity, 2023.
- "Unsupervised Log Parsing and Anomaly Detection with Deep Learning" – Elsevier Neurocomputing, 2020.

6. Environnements de développement et outils complémentaires

Outil	Rôle
VS Code / PyCharm	Développement Python
Docker	Conteneurisation des agents
Jupyter Notebook	Prototypage de modèles IA
Streamlit / Dash	Visualisation des anomalies et interactions utilisateur
Grafana / Kibana	Dashboards avancés (optionnel)

7. Liens GitHub pertinents

Projet	Utilité
logpai/loglizer	Outils de détection d'anomalies sur logs
openai-cyberagent	Prototype d'agent IA appliqué à la sécurité
cyber-agent-framework	Déploiement multi-agent pour détection de comportements
aiops-log-anomaly	Détection IA avancée avec visualisation

✓ 8. Plan de travail détaillé sur 7 mois (par blocs de 2 semaines)

Mois/Semaines	Tâches principales (Milestones)
Mois 1 (S1–S2)	Revue bibliographique sur la détection d'anomalies & logs
Mois 1 (S3–S4)	Collecte et nettoyage de jeux de logs (publics/simulés)
Mois 2 (S5–S6)	Implémentation de l'architecture multi-agents (prototypage)
Mois 2 (S7–S8)	Parsing automatique des logs (agents extracteurs)
Mois 3 (S9–S10)	Implémentation des modèles IA pour détection d'anomalies
Mois 3 (S11–S12)	Test des modèles sur logs simulés (comparaison baseline)
Mois 4 (S13–S14)	Intégration : coordination entre agents + communication
Mois 4 (S15–S16)	Visualisation : création du tableau de bord dynamique
Mois 5 (S17–S18)	Tests sur logs réels, ajustements, détection adaptative
Mois 5 (S19–S20)	Analyse de performance, métriques, visualisation des alertes
Mois 6 (S21–S22)	Rédaction mémoire : chapitres 1 à 4
Mois 6 (S23–S24)	Article 1 IEEE – simulation complète
Mois 7 (S25–S26)	Article 2 + finalisation mémoire + préparation soutenance
Mois 7 (S27–S28)	Relecture, dépôt final, diapos, poster

DETAILS:

Mois 1 : Phase de cadrage et de collecte (S1 à S4)

Semaine 1–2 : Revue bibliographique & définition du problème

- Lecture approfondie sur :
 - journaux systèmes/réseaux,
 - techniques d'analyse de logs,
 - agents intelligents distribués.
- Délimitation du périmètre du projet.
- Début du cahier de charges.

Livrables : plan de lecture, note conceptuelle, mindmap de l'état de l'art.

Semaine 3–4 : Collecte de jeux de données & préparation

- Recherche de datasets : HDFS, BGL, syslog, journaux Linux.
- Installation d'un environnement virtuel Linux/VM pour générer des logs.
- Prétraitement et nettoyage des premiers jeux de données.

Livrables : jeux de données prêts à l'usage + scripts de parsing initial.

Mois 2 : Études comparatives & architecture initiale (S5 à S8)

Semaine 5–6 : Benchmark des techniques de détection d'anomalies

- Implémentation de :
 - méthodes statistiques : z-score, écart interquartile (IQR),
 - IA légère : Isolation Forest, Autoencoder.
- Évaluation de la pertinence pour différents types de journaux.

Livrables : tableau comparatif des méthodes + courbes de performance.

Semaine 7–8 : Conception de l'architecture multi-agents

- Définition des rôles d'agents : collecteur, analyseur, corrélateur, visualiseur.
- Conception UML (diagrammes de composants et séquence).
- Choix des outils : FastAPI, Celery, socket, etc.

Livrables : schéma d'architecture du système + documentation technique.

Mois 3 : Développement initial des agents (S9 à S12)

Semaine 9–10 : Développement des agents de base

- Agent collecteur : parsing automatisé.
- Agent prétraiteur : extraction d'attributs clés.
- Agent analyseur 1 : détection simple avec règles/statistiques.

Livrables : agents exécutables individuellement avec jeux de logs de test.

Semaine 11–12 : Coordination multi-agent + prototypage

- Mise en place d'un orchestrateur (FastAPI ou bus de messages).
- Communication entre agents.
- Détection initiale d'anomalies sur flux simulés.

Livrables : prototype système distribué version 1 + rapport de test interne.

Mois 4 : Ajout de l'IA + visualisation (S13 à S16)

Semaine 13–14 : Intégration de modèles IA dans les agents

- Ajout de modèles IA entraînés (Autoencoder, LSTM léger).
- Intégration dans agent analyseur.
- Test de performance sur log temps réel.

Livrables : performances IA (précision, rappel, F1-score).

Semaine 15–16 : Développement de l’interface web

- Conception UI : Streamlit, Flask, Dash.
- Affichage des anomalies détectées.
- Intégration de filtres, timeline, priorisation.

Livrables : dashboard fonctionnel + test utilisateur (feedback).



Mois 5 : Tests complets, robustesse & scalabilité (S17 à S20)

Semaine 17–18 : Tests sur journaux réels + injection d’anomalies

- Rejeu de logs historiques.
- Simulation d’attaques : brute force, accès root non autorisé, etc.
- Évaluation comportement du système.

Livrables : rapport de robustesse + comparaison avec outils classiques.

Semaine 19–20 : Évaluation technique complète

- Scalabilité : 5 à 10 sources simultanées.
- Latence des agents, consommation CPU/RAM.
- Robustesse à la perte de messages ou logs incomplets.

Livrables : tableau de performance global + visualisation radar.



Mois 6 : Rédaction & valorisation scientifique (S21 à S24)

Semaine 21–22 : Rédaction des chapitres 1 à 4

- Introduction, état de l’art, architecture proposée, implémentation.
- Structuration des annexes, tableaux, schémas.

Livrables : version 1 du mémoire (chap. 1–4) + bibliographie.

Semaine 23–24 : Rédaction du 1er article (conférence IEEE)

- Synthèse scientifique orientée système IA + tests.

- Révision avec encadreur.

Livrables : article prêt à soumettre.



Mois 7 : Finalisation & soutenance (S25 à S28)

Semaine 25–26 : Rédaction des chapitres 5–6 + Article 2 (IEEE Access)

- Résultats, analyse critique, conclusion & perspectives.
- Article 2 orienté architecture distribuée + résilience.

Livrables : mémoire complet version pré-finale + article 2.

Semaine 27–28 : Relecture, présentation & soutenance

- Corrections finales, vérification anti-plagiat.
- Préparation des slides PowerPoint.
- Soutenance blanche (préparation orale).

Livrables : mémoire déposé, présentation prête, soutenance maîtrisée.

✓ 9. Structure détaillée du mémoire

Chapitre 1 : Introduction

- Contexte général
- Problématique
- Objectifs & Contributions
- Méthodologie & Plan du mémoire

Chapitre 2 : État de l'art

- Journaux système & réseau
- Méthodes classiques de détection d'anomalies
- Intelligence artificielle appliquée aux logs
- Systèmes multi-agents intelligents (AI Agents)
- Synthèse comparative

Chapitre 3 : Modèle proposé

- Architecture générale
- Description des agents et de leurs rôles
- Modèles de détection implémentés
- Flot de traitement et diagrammes d'interaction

Chapitre 4 : Implémentation

- Outils utilisés
- Description du code et scripts
- Jeu de données et tests préliminaires
- Problèmes rencontrés

Chapitre 5 : Résultats & Évaluation

- Évaluation quantitative (précision, rappel, F1)
- Évaluation qualitative (visualisation, analyse contextuelle)
- Comparaison avec outils existants

Chapitre 6 : Conclusion & Perspectives

- Résumé des résultats
- Apports réels
- Limites
- Perspectives futures

Annexes

- Codes
- Captures de simulation
- Paramètres expérimentaux

DETAILS:

CHAPITRE 1 : INTRODUCTION GÉNÉRALE

1.1 Contexte général

- L'importance de la cybersécurité dans les infrastructures modernes.
- Rôle central des journaux systèmes et réseaux dans l'analyse post-événement.
- Problème de surcharge de logs + manque de personnel qualifié.

1.2 Motivation du projet

- Explosion du volume de logs générés par les systèmes distribués.
- Besoin d'automatisation et d'intelligence dans l'audit de logs.
- Avantages d'une solution distribuée, autonome et légère.

1.3 Problématique de recherche

- Comment concevoir un système intelligent, distribué, capable de détecter de manière autonome des anomalies dans des journaux hétérogènes, en temps quasi-réel ?

1.4 Objectifs

- Objectif principal + 6 ou 7 objectifs spécifiques détaillés (cf. section 3 du mémoire).

1.5 Méthodologie synthétique

- Démarche itérative : conception → modélisation → implémentation → test → évaluation.
- Méthodologie agile et modulaire basée sur des prototypes.

1.6 Organisation du mémoire

- Brève description des chapitres à venir.

CHAPITRE 2 : ÉTAT DE L'ART ET FONDEMENTS THÉORIQUES

2.1 Typologie des journaux

- Logs systèmes : syslog, journalctl, eventlog
- Logs applicatifs : Apache, NGINX, MySQL, etc.
- Logs réseau : firewall, IDS/IPS (Snort, Suricata)

- Tableau comparatif : format, fréquence, contenu, complexité

2.2 Méthodes classiques de détection d'anomalies

- Détection par seuils fixes ou dynamiques
- Méthodes basées sur les règles (regex, heuristiques)
- Analyse d'entropie, corrélation temporelle

2.3 Intelligence artificielle appliquée à la détection d'anomalies

- IA supervisée : SVM, XGBoost, RF
- IA non-supervisée : Autoencoders, Isolation Forest, LSTM
- Apprentissage en ligne et incrémental

2.4 Architectures multi-agents en cybersécurité

- Définition d'un agent intelligent (autonomie, communication, adaptabilité)
- Architecture MAS (Multi-Agent Systems) : coordination, communication, rôle

2.5 Synthèse critique

- Tableau synthétique des forces/faiblesses de chaque méthode
- Justification du choix d'une architecture distribuée IA à base d'agents

CHAPITRE 3 : CONCEPTION DE L'ARCHITECTURE MULTI-AGENTS IA

3.1 Hypothèses de conception

- Type de logs disponibles
- Capacité machine (RAM, CPU, bande passante)
- Fréquence de génération d'événements

3.2 Spécification des agents

Agent	Rôle principal	Entrée	Sortie	IA intégrée
Collecteur	Capture de journaux	Fichiers logs	Log brut	Non
Parseur	Extraction & structuration	Log brut	JSON	Non
Détecteur	Analyser et signaler	JSON	Alertes	Oui
Corrélateur	Croisement temporel	Alertes multiples	Alertes corrélées	Oui/Non
Visualiseur	Affichage utilisateur	Alertes	UI	Non

3.3 Modèle logique global

- Diagrammes de séquence (cas d'usage)
- Schéma d'interaction agent ↔ agent
- Synchronisation entre processus

3.4 Architecture logicielle et technique

- Technologies retenues (Python, FastAPI, SocketIO, etc.)
- Composants logiciels (API, bus de messages, base NoSQL)

3.5 Stratégie de communication

- Protocole REST, MQTT ou sockets
- Gestion des erreurs, des interruptions



CHAPITRE 4 : MISE EN ŒUVRE TECHNIQUE

4.1 Outils et environnement

- Langages : Python 3.x
- IDE : VS Code, PyCharm
- Outils : Docker, Jupyter, Streamlit
- Bibliothèques : Scikit-learn, Keras, pandas, logparser

4.2 Implémentation des agents

- Code-source (résumé), architecture de chaque script
- Fonctionnement isolé et en coordination

4.3 Données d'expérimentation

- Description des datasets (HDFS, BGL, logs simulés)
- Statistiques sur le contenu : volume, nombre d'entrées, durée

4.4 Intégration globale du système

- Assemblage des agents
- Gestion des flux de données entre modules
- Gestion de la journalisation des alertes

4.5 Interface de visualisation

- Présentation de l'interface (capture d'écran, schéma)
- Fonctionnalités : filtrage, tri, exportation, notification

CHAPITRE 5 : ÉVALUATION EXPÉRIMENTALE

5.1 Plan d'évaluation

- Scénarios testés (réel, simulé, injecté)
- Critères : précision, rappel, latence, charge système

5.2 Méthodes d'évaluation

- Analyse comparative : IA vs règles fixes
- Mesures d'erreur : confusion matrix, courbes ROC

5.3 Résultats expérimentaux

- Tableaux et graphes : performance par agent, par log
- Détail des cas d'anomalies détectés

5.4 Discussion critique

- Analyse des réussites et des échecs
- Analyse des faux positifs et faux négatifs

5.5 Analyse de scalabilité et robustesse

- Résistance à la montée en charge (nombre d'agents)
- Résilience en cas de panne d'un agent

CHAPITRE 6 : CONCLUSION ET PERSPECTIVES

6.1 Résumé des contributions

- Système multi-agent innovant
- IA légère et adaptable intégrée aux journaux
- Interface utilisateur fonctionnelle

6.2 Limites identifiées

- Dépendance aux logs disponibles
- Difficultés de généralisation

6.3 Perspectives

- Intégration de LLMs (GPT/Claude) pour analyse sémantique des logs
- Passage à une architecture cloud-native distribuée

- Extension vers un produit open-source complet
-

Annexes

- Dictionnaire des variables
 - Exemples de logs bruts vs normalisés
 - Documentation technique des agents (API, endpoints)
 - Résultats bruts des tests
 - Fichiers de configuration (Docker, .env, etc.)
-

Bibliographie

- Format IEEE
- Articles scientifiques, normes, thèses, livres
- Liens GitHub ou pages officielles des outils utilisés

✓ 10. Articles scientifiques issus du mémoire

ARTICLE 1 – Pour conférence IEEE (type IJCNN ou ICMLA)

a) Titre

Distributed Intelligent Agents for Anomaly Detection in System Logs: A Lightweight Cybersecurity Framework for SMEs

Mots-clés : anomaly detection, log parsing, AI agents, cyber-audit, lightweight AI, system monitoring.

b) Questions de recherche

1. Quelle est la performance d'un système multi-agents IA pour la détection d'anomalies logiques ?
2. Comment répartir efficacement les tâches entre agents autonomes ?
3. Quels modèles IA légers conviennent aux environnements à ressources limitées ?
4. Le système peut-il être auto-adaptatif ?
5. Quel est l'impact du volume/format des logs sur la détection ?
6. Comment le système se compare-t-il aux outils traditionnels ?

c) Abstract

Cet article présente un cadre intelligent distribué pour l'analyse autonome de journaux systèmes. Basé sur des agents IA spécialisés, le système détecte en temps réel des anomalies dans divers types de logs (réseau, système, application). L'architecture légère permet une détection rapide, contextualisée, et visualisable. Des tests sur des datasets publics montrent une amélioration significative de la précision par rapport aux outils traditionnels tout en restant adapté aux infrastructures limitées.

d) Structure

1. Introduction
2. Related Work
3. Architecture Multi-Agent Proposée
4. Méthodes d'Apprentissage Utilisées
5. Simulation et Résultats
6. Évaluation et Discussion
7. Conclusion et perspectives

ARTICLE 2 – Pour IEEE Access (plus technique et approfondi)

a) Titre

Adaptive Multi-Agent AI Framework for Real-Time Log Anomaly Detection in Distributed Systems

Mots-clés : multi-agent system, adaptive AI, log anomaly detection, unsupervised learning, intelligent security framework.

b) Questions de recherche

1. Comment concevoir un système intelligent auto-apprenant pour logs réseau ?
2. Quelle architecture multi-agent garantit la robustesse en environnement dynamique ?
3. Quelle combinaison de modèles IA offre la meilleure efficacité ?
4. Comment assurer la scalabilité sans infrastructure lourde ?
5. Peut-on généraliser l'approche à d'autres formats de logs ?

c) Abstract

Dans cet article, nous proposons une architecture multi-agents intelligente pour la détection adaptative d'anomalies dans les journaux systèmes. Chaque agent, spécialisé, est capable de

traiter, analyser et apprendre de façon autonome. L'approche proposée utilise des modèles non supervisés et des mécanismes de communication inter-agent pour maintenir une vision cohérente des événements système. Les résultats sur des logs réels et synthétiques montrent une détection rapide, robuste et adaptée aux environnements distribués.

d) Structure

1. Introduction et Motivation
2. Fondements de l'architecture multi-agent
3. Méthodologie (données, IA, orchestration)
4. Études de cas & Scénarios simulés
5. Résultats expérimentaux (performance, latence)
6. Discussion approfondie (limites, évolutivité)
7. Conclusion et Recommandations

DETAILS:



ARTICLE 1 : Pour une conférence IEEE (IJCNN, ICMLA, ou AICCSA)

Titre proposé :

Conception d'un Système Multi-Agents Légers pour la Détection Autonome d'Anomalies dans les Journaux Systèmes : Une Approche IA Distribuée pour la Cybersécurité



Mots clés :

anomalie dans les logs, intelligence artificielle, agents intelligents, cybersécurité distribuée, détection non supervisée, architecture multi-agent

? Questions de recherche

1. Comment concevoir une architecture modulaire d'agents IA coopérants pour l'analyse de journaux systèmes ?
2. Quels types d'agents sont nécessaires pour effectuer un audit automatisé ?
3. Quelles interactions entre les agents assurent la détection efficace ?
4. Quels sont les avantages d'un système multi-agent face aux approches monolithiques ?
5. Quelle est la charge système induite par cette architecture distribuée ?



Résumé (Abstract)

Dans ce papier, nous présentons une architecture distribuée à base d'agents intelligents pour la détection d'anomalies dans les journaux systèmes et réseaux. Chaque agent est dédié à une tâche spécifique (collecte, parsing, détection, corrélation, visualisation), formant un système modulaire, évolutif et résilient. Le système utilise des modèles IA légers intégrés localement dans les agents, assurant une autonomie dans l'analyse. L'interaction entre agents permet une compréhension globale des événements. L'approche est testée sur plusieurs jeux de logs (HDFS, BGL, journaux Linux simulés). Les résultats montrent une capacité de détection comparable à celle de systèmes centralisés tout en garantissant flexibilité, scalabilité, et adaptation aux environnements à ressources limitées.



Structure détaillée de l'article

1. Introduction

- Contexte : surcharge des logs, besoin d'autonomie
- Limites des approches traditionnelles
- Objectif et contributions principales

2. Revue de littérature

- Détection d'anomalies dans les logs : méthodes classiques vs IA
- Systèmes multi-agents en cybersécurité
- Travaux existants et leurs limites

3. Conception du système proposé

- Définition des agents
- Architecture logique et technique
- Flux de données et coordination agentique

4. Implémentation

- Technologies : Python, Scikit-learn, SocketIO
- Rôles et scripts de chaque agent
- Exemples de scénarios simulés

5. Étude de cas et évaluation initiale

- Datasets utilisés
- Résultats de détection (faux positifs, taux de détection)
- Analyse de la charge et du comportement inter-agent

6. Discussion

- Interprétation des performances
- Avantages de la modularité
- Limites identifiées

7. Conclusion et travaux futurs

- Synthèse des résultats
- Ouverture vers systèmes hybrides (LLM + agents) ou IoT



ARTICLE 2 : Pour le journal de classe Q1 de nom „**IEEE Access**“ (revue indexée, open access)

Titre proposé :

Détection Adaptative et Scalabilité dans un Système Multi-Agent pour l'Audit Sécuritaire des Journaux Systèmes : Évaluation Approfondie et Déploiement Expérimental

Mots clés :

log analysis, anomaly detection, adaptive AI, cybersecurity automation, lightweight agents, real-time monitoring

Questions de recherche

1. Quels modèles d'IA légers sont les plus efficaces pour la détection d'anomalies dans les logs ?
2. Le système peut-il s'adapter automatiquement à de nouvelles sources de journaux ?
3. Quelles performances obtient-on sur différents jeux de données publics et simulés ?
4. Quel est le niveau de scalabilité du système dans un environnement multi-source ?
5. Comment se comporte le système sous contraintes : pannes, surcharge, bruit ?
6. Quel est le gain par rapport à des outils traditionnels d'analyse de logs ?



Résumé (Abstract)

Ce travail présente une évaluation approfondie d'un système distribué basé sur des agents intelligents légers, déployé pour détecter des anomalies dans des journaux système et réseau. Le système repose sur des agents autonomes spécialisés intégrant des modèles IA adaptatifs non supervisés. L'article présente une analyse comparative de plusieurs modèles (Isolation Forest, Autoencoder, LSTM) et leur intégration dans le système. Une série d'expériences sont menées sur des jeux de données variés, incluant des logs réels, simulés et bruités. L'étude mesure la précision, le rappel, la latence et la robustesse du système, en conditions contrôlées et en montée en charge. Les résultats démontrent la viabilité de cette approche distribuée pour des applications concrètes en cybersécurité.



Structure détaillée de l'article

1. Introduction

- Problème : surcharge, bruit, hétérogénéité des logs
- Objectif : audit intelligent, scalable, modulaire
- Contributions de l'article

2. Système multi-agent proposé

- Architecture complète
- Logique d'adaptation locale
- Synchronisation et message passing

3. Méthodologie d'évaluation

- Datasets : HDFS, BGL, logs injectés
- Métriques : Précision, rappel, F1-score, temps de traitement
- Cas de panne, cas de surcharge, cas d'erreur syntaxique

4. Résultats expérimentaux

- Comparaison des modèles IA
- Taux d'anomalies détectées selon la charge
- Évolution du système avec ajout de sources

5. Discussion approfondie

- Scalabilité observée
- Robustesse en cas de défaillance

- Intégration avec des outils tiers (ElasticSearch, Kibana)

6. Perspectives

- Interopérabilité avec des LLMs
- Déploiement réel (université, PME)
- Renforcement via apprentissage actif

7. Conclusion

- Synthèse des performances
- Feuille de route pour industrialisation